

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering (CSE)**MID SEMESTER EXAMINATION****WINTER SEMESTER, 2023-2024****DURATION: 1 HOUR 30 MINUTES****FULL MARKS: 75****CSE 4303: Data Structures**

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer all 3 (three) questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

1. a) i. Determine the output for the program shown in Code Snippet 1.

4

(CO1)

(PO1)

```

1 #include<iostream>
2 #include<stack>
3 int main(){
4     std::stack<int> s;
5     int n = 50;
6     while(n>0){
7         s.push(n%2);
8         n = n/2;
9     }
10    while(!s.empty()){
11        std::cout<<s.top();
12        s.pop();
13    }
14 }
```

1 3 0 0 2 5 0
2 5
1 2
6
3
1

Code Snippet 1: C++ program involving push and pop operations of a stack for Question 1.a)i

- ii. Use a stack to evaluate the following postfix expression and show the state of the stack after processing each token:

5

(CO2)

(PO1)

2 7 1 - * 8 4 / - 4 *

- b) Suppose you are given a single linked list
- L
- which is an object of
- `SingleList`
- class and it contains
- n
- integers. Each of these integers are stored in individual objects of
- `Node`
- class. You are provided with a function to copy the contents of the linked list into an array called
- `arr`
- , as shown in Code Snippet 2. Assume that the array has been declared globally with enough space to contain all elements of
- L
- . The
- `SingleList`
- class has a public method called
- `size()`
- which iterates over the elements in
- L
- until it encounters
- `nullptr`
- and returns the total number of elements.

4 + 4

(CO3)

(PO2)

Determine the time complexity of this function. Also explain the potential problems with this implementation and possible ways to improve its time complexity.

```

1 void copyList(SingleList L, int arr[])
2 {
3     Node* ptr = L.begin();
4     for (int i = 0; i < L.size(); i++)
5     {
6         arr[i] = ptr->value();
7         ptr = ptr->next();
8     }
9 }
```

Code Snippet 2: C++ code for Question 1.b that copies a linked list into an array

- c) Suppose you have built a class named 'Queue' that implements all the core functionalities of a Queue data structure. Describe how you can simulate the push and pop operations of a Stack using the Queue class without creating any additional data structures.

8
(CO2)
(PO1)

2. a) Figure 1 represents a B+ tree where the maximum number of pointers per node and the maximum number of records in the leaf node are denoted as M and L , respectively.

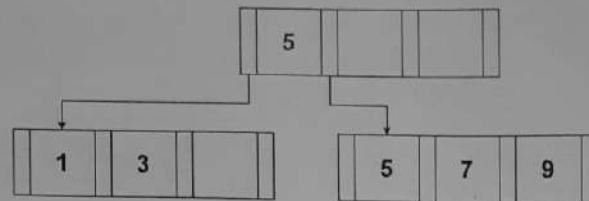


Figure 1: B+ Tree containing five integers for Question 2.a

Considering $M = 4$ and $L = 3$, answer the following questions:

- i. Insert the following integers into the B+ Tree sequentially and show the state of the B+ Tree after each insertion:
2, 4, 6, 8, 10
- ii. After performing all the insertions in Question 2.a)i, delete the following integers from the B+ Tree sequentially and show the B+ Tree after each deletion:
9, 7, 8, 3
- b) Insert the following sequence of values into an empty AVL tree and show the state of the tree after each insertion:

5
(CO2)
(PO1)

4
(CO2)
(PO1)

9
(CO2)
(PO1)

- c) Determine the runtime complexities of each of the two functions given in Code Snippet 3 providing necessary justifications.

7
(CO2)
(PO1)

```

1 void function1(int n)
2 {
3     for(int i = n; i > 0; i=i/2)
4         std::cout << i << endl;
5 }
6
7 void function2(int n)
8 {
9     for (int i = 0; i < n; i++){
10         if(i>10)
11             break;
12         else
13             std::cout << i << endl;
14     }
15 }

```

Code Snippet 3: C++ code containing two functions for Question 2.c

3. a) Suppose you are given a Binary Tree of characters whose preorder and inorder traversals are given as follows:
 Preorder Traversal: A B D H I E C F J G K L
 Inorder Traversal: H D I B E A F J C K G L
 Illustrate the Binary Tree and find its postorder traversal.

7
(CO1)
(PO1)

- b) Consider a Binary Tree containing a string in each node as shown in Figure 2. Illustrate the result after transforming this Binary Tree into a Binary Search Tree of height 2 using the lexicographical ordering of the strings.

8
(CO2)
(PO1)

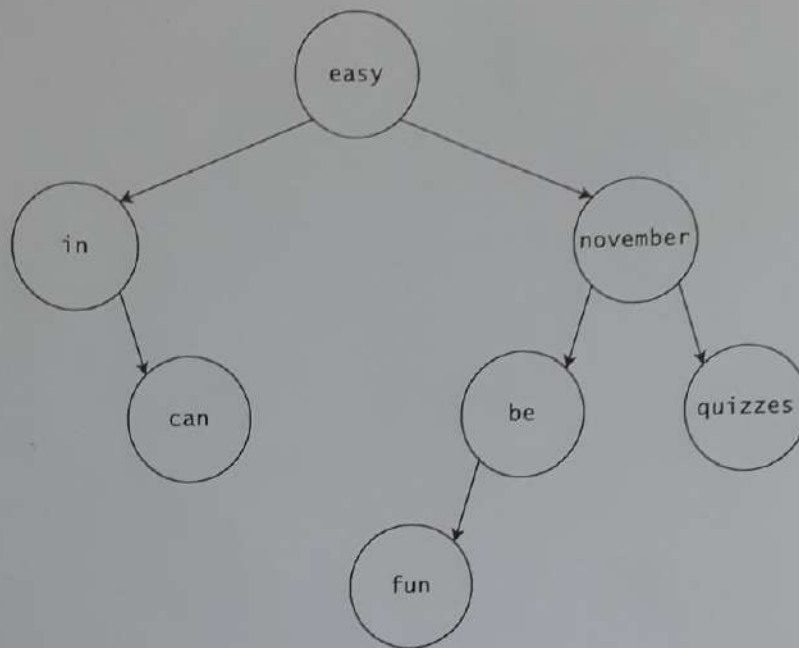


Figure 2: A sample Binary Tree containing strings Question 3.b

- c) Design a C++ function that takes two parameters where the first parameter is an object of `BinaryTree` class and the second parameter is an integer, `targetSum`. The function should return `true` if there exists a root-to-leaf path in the Binary Tree such that adding up all the values along that path equals `targetSum`, and `false` otherwise. For instance, if the function is called for the Binary Tree shown in Figure 3 with `targetSum` set to 20, it should return `true` since the path: 8-6-1-3-2 adds up to 20.

10
(CO2)
(PO1)

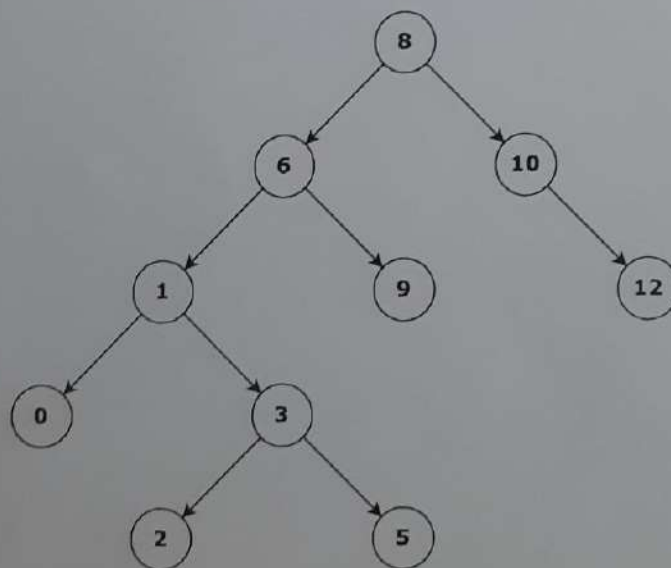


Figure 3: A sample Binary Tree containing 10 integers for Question 3.c